The Ch(e)at Code

Agent Patterns for Copilot Chat

↑ ↑ ↓ ↓ ← → ← → B A START

ISSUE #011 • JUNE 9, 2026 • 🔧 PRACTICAL BUILD

Last issue, we named the Multi-Agent Handoff pattern. We mapped three transfer models — cold, warm, and hot — and drew the trust boundaries that keep a multi-agent system coherent. This issue, we build it. One session, two agents, context that carries across.

This issue's build: A warm-transfer handoff between an HR agent and an IT Support agent in Copilot Studio, with Dataverse session state for analytics and Power Automate for out-of-scope detection — the practical implementation of Issue #010's conceptual framework.

🎮 AGENT SPOTLIGHT

The Two-Agent Handoff

HR Knowledge Agent ↔ IT Support Agent • Copilot Studio + Dataverse + Power Automate

The scenario: An enterprise has two Copilot Studio agents — one for HR (PTO, benefits, onboarding) and one for IT Support (devices, access, software). An employee asks the HR agent: "I'm onboarding a new hire next week — how do I get them a laptop?" The HR agent knows about onboarding checklists, but laptop provisioning is IT's domain. Today, the agent either hallucinates an answer or says "contact IT." Neither is acceptable.

The solution: When the HR agent detects an out-of-scope query, it generates a conversation summary via Azure OpenAI, writes a handoff record to Dataverse, and transfers the user to the IT agent using Copilot Studio's agent transfer topic. The IT agent reads the Dataverse record, injects the context, and picks up mid-conversation. The user never repeats themselves.

The stack: Copilot Studio agent transfer + Dataverse `AgentSession` table + Power Automate for classification & logging. Five components, all buildable in a single session.

🧩 **PATTERN BREAKDOWN**

Warm Handoff Build: Five Components

Copilot Studio agent transfer + Dataverse session state + Power Automate classification & analytics

Why this matters: The warm transfer is the right default for most multi-agent deployments (see Issue #010). It gives users a seamless experience without the architectural complexity of shared session state. This build shows you every piece in order — so you can demo it to a customer and hand them a repeatable implementation pattern.

END-TO-END BUILD

```
User asks HR Agent an IT question
  ↓
Fallback topic → low confidence → triggers classification
  ↓
Power Automate → calls Azure OpenAI: "Is this HR or IT?"
  ↓
If IT: generate conversation summary → write to AgentSession
  ↓
Agent Transfer topic → transfer to IT Agent
  ↓
IT Agent greeting topic → reads AgentSession → injects context
  ↓
Seamless continuation with full context
```

Five components, in build order:

1. **Two agents with distinct scopes.** Agent A: HR Knowledge Agent (PTO, benefits, onboarding). Agent B: IT Support Agent (devices, access, software). Each has its own knowledge sources, topics, and system prompt. The critical

line in each system prompt: *"If the user asks about [other domain], do not attempt to answer — trigger the handoff topic."* This prevents hallucination across domain boundaries and is the foundation everything else builds on.

2. **Out-of-scope detection topic.** In each agent, build a fallback topic that fires when generative answers return low confidence. This topic calls a Power Automate flow that classifies the query: *"Is this question about HR or IT?"* If the query belongs to the other agent, the flow returns a signal to trigger the handoff. Don't rely on keyword matching — use Azure OpenAI for classification so it handles ambiguous queries like "new hire laptop" that span both domains.

3. **Copilot Studio agent transfer with summary.** Use the "Transfer to agent" system topic. Before the transfer fires, generate a conversation summary via Azure OpenAI: *"Summarize this conversation in 3 sentences, including the user's original question and what's been discussed so far."* Pass the summary as a context variable to the receiving agent. This is the warm transfer — Agent B starts with context instead of "How can I help you?"

4. **Dataverse session state.** Create an `AgentSession` table with columns: `session_id`, `user_id`, `source_agent`, `target_agent`, `summary`, `handoff_reason`, `timestamp`. On handoff, write the session record. On receive, the greeting topic queries Dataverse for the session and injects the summary as context. This is what turns a one-off transfer into a system — every handoff is logged, queryable, and analyzable.

5. **Handoff analytics.** Power Automate logs every transfer. Build a Power BI dashboard tracking: handoff volume by direction (HR→IT vs IT→HR), round-trip rate (user bounced back to original agent), and resolution rate per agent. Set an alert: *"HR→IT handoff rate spiked 40% this week"* — that signals a knowledge gap in the HR agent that needs fixing. Analytics close the loop and tell you where to invest in each agent's knowledge base.

> **Key insight:** The handoff analytics are the most valuable component long-term. They tell you which agents need broader knowledge, which handoff paths are friction points, and where misroutes are happening. Without analytics, you're flying blind — you built a multi-agent system but can't tell if it's working. With analytics, every handoff becomes a data point that improves the system.

> ⚠ **Watch for circular handoffs:** If both agents have weak coverage in an overlapping area (e.g., "software licenses for new hires"), users can get bounced between agents indefinitely. Add a `handoff_count` variable that increments on each transfer. If it exceeds 1 in a session, escalate to a human instead of transferring again. The Dataverse `AgentSession` table makes this trivial to detect and alert on.

🛠️ **TRY THIS NOW**

Build a Two-Agent Handoff in One Session

1. **Clone an existing agent and split the knowledge.** Take one of your customer's working Copilot Studio agents and duplicate it. Strip the knowledge sources: give Agent A the HR docs, give Agent B the IT docs. Update each system prompt with the boundary instruction: *"If the user asks about [other domain], do not answer — trigger the handoff topic."* Two agents, ten minutes.

2. **Add a fallback topic that detects out-of-scope queries.** In each agent, create a topic triggered by the system's low-confidence fallback event. Wire it to a Power Automate flow that classifies the query using Azure OpenAI. If the query belongs to the other domain, set a variable that triggers the transfer. Test with: *"How do I get a laptop for my new hire?"* in the HR agent.

3. **Configure agent transfer with conversation summary passing.** In the handoff topic, add an Azure OpenAI action that summarizes the conversation. Store the summary in a context variable and configure the "Transfer to agent" node pointing at Agent B. On the receiving side, build a greeting topic that reads the context variable and starts with: *"I see you were asking about [summary]. Let me help with that."*

4. **Test the full loop.** Ask Agent A a question that belongs to Agent B. Verify: (a) the fallback fires, (b) the classification is correct, (c) the summary is generated, and (d) Agent B picks up with full context. Then try the reverse — ask Agent B an HR question. Both directions should work identically.

💡 **The line that opens doors:** "What if your employees could ask any agent any question and always get routed to the right one — without restarting the conversation?"

Got an agent or pattern to share?

Reply to this email and you might be featured in a future issue.

The Ch(e)at Code

AIWF Team • AI Business Solutions

Questions or ideas for the next issue? Reply to this email.

Microsoft Internal • Not for external distribution